



Smart Library Management System (SLMS)

Project Report

Course: Java in Programming

Course ID: CSE2006

Slot:-C12+F14

Submitted by:- Eshant Baranwal

Reg No.:- 24BCY10212

1. Introduction

The Smart Library Management System (SLMS) is a console-based Java application built to manage the day-to-day operations of a small library: cataloging books, registering members and staff, issuing and returning books, calculating overdue fines, and generating operational reports. The project was undertaken as the open-ended "Build Your Own Project" submission for the Java Programming course, and was deliberately scoped to exercise the full range of core Java and object-oriented concepts covered in the syllabus: abstraction and inheritance, polymorphism, interfaces, the Collections Framework, custom checked exceptions, file I/O, and classic design patterns (Singleton, Factory, and Data Access Object).

Rather than relying on an external database engine, the system persists its data to plain CSV files behind a DAO abstraction layer. This keeps the project dependency-free and trivially runnable on any machine with a JDK installed, while still demonstrating the same layered architecture (UI → Service → DAO → Storage) that a production system backed by a real database would use.

2. Problem Statement

Small and mid-sized libraries often rely on manual registers or spreadsheets to track book inventory and borrowing activity. This is error-prone (duplicate entries, lost records), makes it difficult to answer simple analytical questions such as "which books are overdue right now?" or "which titles are most popular?", and provides no separation of access between library staff and general members. SLMS addresses this by providing a single, role-aware application that models the library's real workflow as a set of cleanly separated software modules.

Target users:

- **Admin** - full system access: manages staff accounts, the full catalog, all transactions, and every analytical report.
- **Librarian** - day-to-day operator: manages the book catalog and issue/return transactions, and views operational reports.
- **Member** - the borrower: registers their own account, browses/searches the catalog, borrows/returns books, and views their own history and fines.

3. Functional Requirements

Module 1: User & Access Management

Registration (self-service for Members; Admin-created for Librarians/Admins), SHA-256-hashed authentication, and role-based menu access enforced via an abstract User class with three polymorphic subclasses.

Module 2: Book Catalog & Inventory Management

Add, update, remove, list, and search books by title, author, or category, with ISBN-uniqueness validation and live available-copy tracking.

Module 3: Issue/Return & Fine Management

Issue a book to a member (enforcing one active loan per member per title and a 14-day due date), return a book, and automatically compute a per-day late fine if returned after the due date.

Module 4: Reporting & Analytics

Inventory summary, most-borrowed-books ranking, overdue-items report, and per-member borrowing activity/fines report.

Input/Output structure: the application is menu-driven over the console. Each menu option prompts for the specific fields it needs (e.g. title/author/ISBN when adding a book) and prints either a confirmation with the created/updated record, or a specific error message drawn from a custom exception.

4. Non-Functional Requirements

Requirement	How it's addressed
Performance	In-memory list operations over small CSV datasets; DAO interfaces isolate storage so it can be swapped
Security	Passwords are SHA-256 hashed before storage; role-based menus prevent Members from reaching Admin
Reliability	Every service method validates input and fails predictably via typed exceptions instead of crashing; each file
Scalability	Layered architecture (UI -> Service -> DAO -> Storage) means the CSV engine can be replaced by JDBC/
Maintainability	Clear package-by-layer structure, consistent naming, and Javadoc-style comments make the codebase easy
Error handling	Six custom checked exceptions communicate exactly what went wrong; the UI layer catches each one and
Logging/Monitoring	A Singleton AppLogger timestamps every significant action and error to logs/app.log.
Resource efficiency	Files are opened, read/written, and closed per operation via try-with-resources; no persistent open handles

5. System Architecture

SLMS follows a classic layered architecture. The Presentation layer (ConsoleUI) never touches storage directly; every request flows down through the Service layer (business rules and validation) and the DAO layer (persistence), and every failure flows back up as a typed exception. Cross-cutting concerns (logging, file access, validation, ID generation) are centralized in the util package so they are implemented once and reused everywhere.

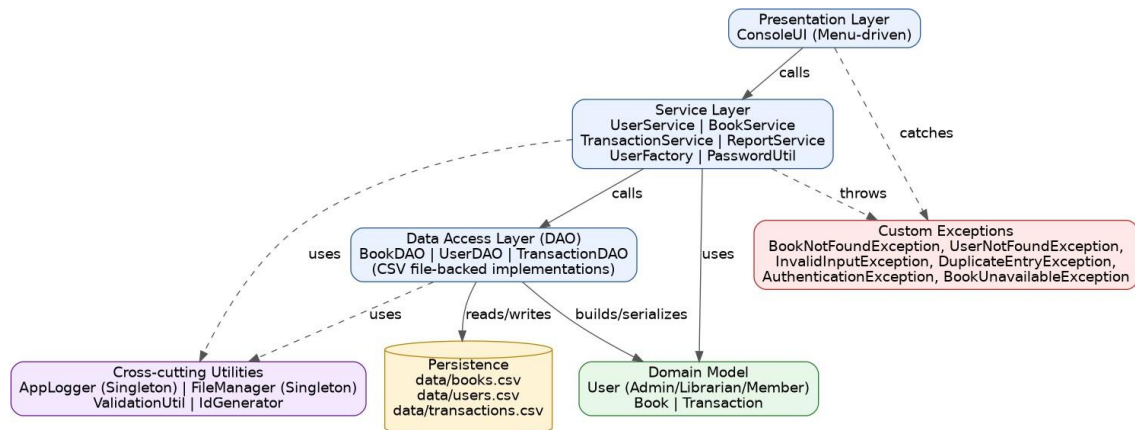


Figure 1: Layered system architecture.

6. Design Diagrams

6.1 Use Case Diagram

Three actors interact with the system, each scoped to the use cases relevant to their role, enforced in ConsoleUI's role dispatch logic.

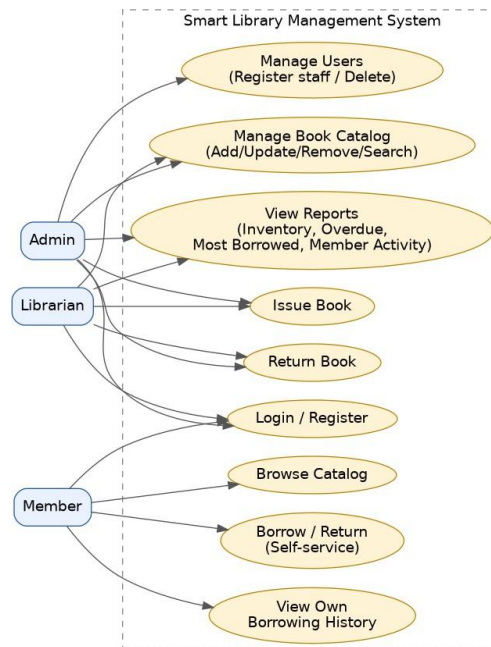


Figure 2: Use case diagram.

6.2 Process Flow / Workflow Diagram

The end-to-end flow from application start, through the role-specific menu, back to logout and the main menu.

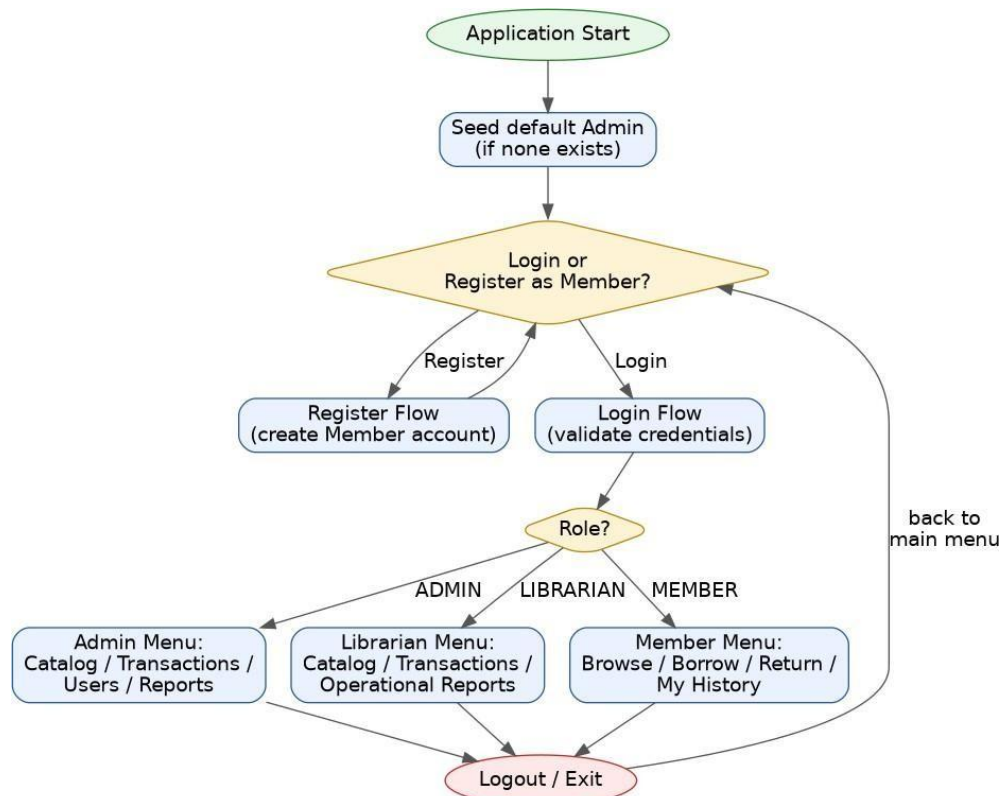


Figure 3: Process flow / workflow diagram.

6.3 Sequence Diagram - Issue Book

Message flow when a Librarian or Member issues a book: the TransactionService checks availability and any existing active loan before creating a new transaction and decrementing the book's available copies.

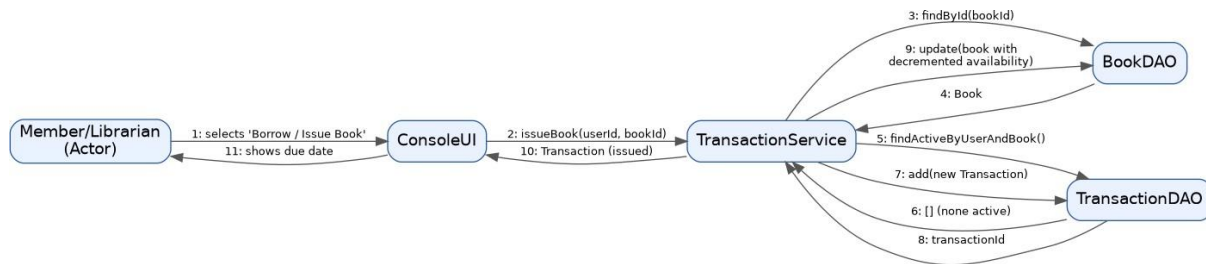


Figure 4: Sequence diagram for the issue-book flow.

6.4 Class / Component Diagram

Key classes and their relationships: an abstract User with three role subclasses, the Book and Transaction entities, the service layer that orchestrates them, and the UserFactory responsible for instantiating the correct User subtype.

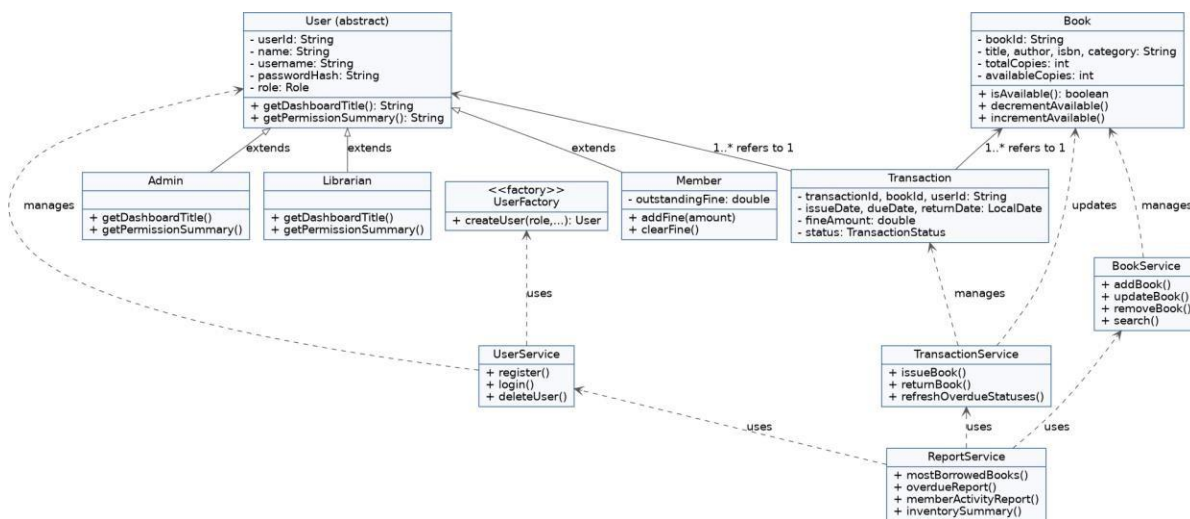


Figure 5: Class diagram.

6.5 ER / Storage Design Diagram

The system persists three CSV-backed "tables". TRANSACTIONS is the associative entity linking USERS and BOOKS (many issue/return records per user and per book).

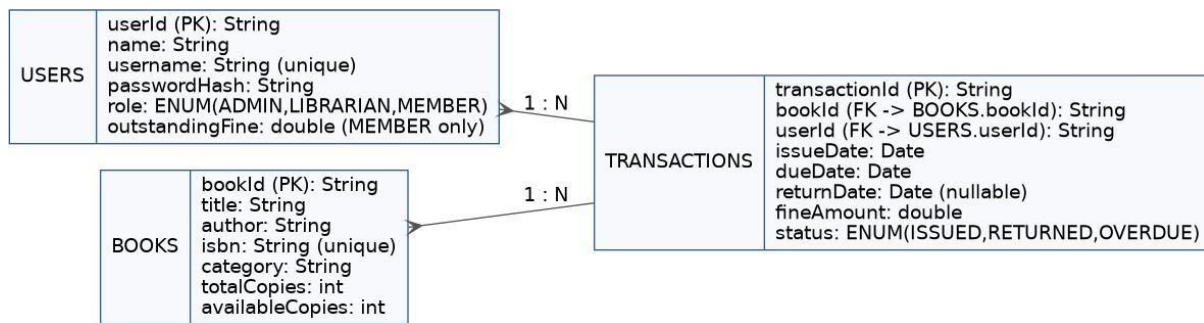


Figure 6: Entity-relationship / storage design diagram.

7. Design Decisions & Rationale

CSV files instead of a database. The development environment had no access to Maven Central or a database driver repository. Rather than compromise on a fragile setup, storage was abstracted behind DAO interfaces so a future JDBC-backed implementation can be swapped in without touching the service or UI layers.

Abstract User with role subclasses. Admin, Librarian, and Member share common fields (id, name, credentials) but differ in permissions and dashboard behaviour. Modeling this with inheritance and polymorphism (getDashboardTitle(), getPermissionSummary()) avoids a tangle of if/else role checks scattered through the UI.

Factory pattern for user creation. UserFactory centralizes the decision of which User subclass to instantiate based on a Role, so calling code (UserService, UserDAOImpl) never needs a switch statement of its own.

Singleton Logger and FileManager. Both need to coordinate access to a single shared resource (a log file, and the data directory) across the whole application; a Singleton guarantees one shared instance without passing references through every constructor.

Custom checked exceptions over generic RuntimeExceptions. Checked exceptions force calling code (the UI layer) to explicitly handle each failure mode, which maps directly onto user-facing error messages instead of uncaught stack traces.

In-memory fake DAOs for testing. With no internet access to download JUnit/Mockito from Maven Central, the test suite instead implements the same DAO interfaces with simple in-memory Maps. This keeps tests fast, deterministic, and fully isolated from the filesystem.

8. Implementation Details

Package structure (com.slms):

- **model** - Role, TransactionStatus (enums); User (abstract), Admin, Librarian, Member, Book, Transaction.

- **exception** - BookNotFoundException, UserNotFoundException, InvalidInputException, DuplicateEntryException, AuthenticationException, BookUnavailableException.
- **util** - AppLogger and FileManager (Singletons), ValidationUtil, IdGenerator.
- **dao** - BookDAO/UserDAO/TransactionDAO interfaces with CSV-file-backed implementations.
- **service** - UserService, BookService, TransactionService, ReportService, UserFactory, PasswordUtil.
- **ui** - ConsoleUI, the single menu-driven front end.

The project totals 32 main-source Java files plus 8 test-source files, organized with a clean one-class-per-concern style. All persistence goes through FileManager, which exposes generic readLines/writeAll/appendLine operations so each DAO only needs to know its own CSV schema and row-parsing logic. Passwords are hashed with SHA-256 via PasswordUtil before ever reaching disk. The default administrator account (username: admin, password: admin123) is seeded automatically on first run if no ADMIN user exists yet.

Fine calculation logic (TransactionService.returnBook): the number of days between the due date and the actual return date is computed with java.time.temporal.ChronoUnit, floored at zero for on-time returns, and multiplied by a fixed FINE_PER_DAY rate defined as a constant on the Transaction class.

9. Screenshots / Results

Below is a captured console transcript of a full sample session: registering a member, logging in as the seeded Admin, adding a book to the catalog, issuing it to the member, checking the inventory summary, returning the book, and viewing the most-borrowed-books report.

```
[First run] Created default admin -> username: admin, password: admin123
=====
Smart Library Management System (SLMS)
=====

1. Login
2. Register as Member
3. Exit
Choose: Full name: Choose a username: Choose a password: Registered successfully. Your member id is U002

1. Login
2. Register as Member
3. Exit
Choose: Username: Password: Welcome, System Admin (Administrator Dashboard)

--- Admin Menu (System Admin) ---
1. Manage Catalog
2. Manage Transactions
3. Manage Users
4. Reports
5. Logout

-- Catalog Management --
1. Add Book
2. Update Book
3. Remove Book
4. List All Books
5. Search Books
6. Back
Title: Author: ISBN (10 or 13 digits): Category: Total copies: Added: [B001] Clean Code by Robert C. Martin (Software

--- Admin Menu (System Admin) ---
1. Manage Catalog
2. Manage Transactions
3. Manage Users
4. Reports
5. Logout

-- Transaction Management --
1. Issue Book to Member
2. Return Book
3. List All Transactions
4. Overdue Report
5. Back
Member user id: Book id: Issued. Txn[T001] book=B001 user=U002 issued=2026-09-15 due=2026-09-29 returned=- fine=0.00

--- Admin Menu (System Admin) ---
1. Manage Catalog
2. Manage Transactions
3. Manage Users
4. Reports
5. Logout

-- Reports & Analytics --
1. Inventory Summary
2. Most Borrowed Books
3. Overdue Report
4. Member Activity
5. Back
Titles: 1 | Total copies: 3 | Available: 2 | On loan: 1

--- Admin Menu (System Admin) ---
1. Manage Catalog
2. Manage Transactions
```

3. Manage Users
4. Reports
5. Logout

-- Transaction Management --

1. Issue Book to Member
2. Return Book
3. List All Transactions
4. Overdue Report
5. Back

Transaction id to return: Returned. Fine due: 0.0

--- Admin Menu (System Admin) ---

1. Manage Catalog
2. Manage Transactions
3. Manage Users
4. Reports
5. Logout

-- Reports & Analytics --

1. Inventory Summary
2. Most Borrowed Books
3. Overdue Report
4. Member Activity
5. Back

1x [B001] Clean Code by Robert C. Martin (Software Engineering) - 3/3 available

--- Admin Menu (System Admin) ---

1. Manage Catalog
2. Manage Transactions
3. Manage Users
4. Reports
5. Logout

1. Login
2. Register as Member
3. Exit

Choose: Goodbye!

10. Testing Approach

Because the build environment has no internet access to Maven Central (so JUnit/Mockito cannot be downloaded), the project ships a small, dependency-free test harness (com.slms.test.Assert) offering assertTrue, assertEquals, and assertThrows helpers, plus hand-written in-memory implementations of each DAO interface (InMemoryBookDAO, InMemoryUserDAO, InMemoryTransactionDAO) so tests never touch the filesystem and remain fast and deterministic.

Coverage:

- **BookServiceTest** - rejects empty title / invalid ISBN / non-positive copies; verifies available copies equal total on creation; rejects duplicate ISBN; verifies update and search behaviour; verifies lookup fails after removal.
- **UserServiceTest** - verifies role assignment on registration; rejects duplicate usernames; verifies successful login and its rejection for a wrong password or an unknown username; confirms the password is never stored in plaintext.
- **TransactionServiceTest** - verifies the 14-day due-date calculation; verifies available copies decrease on issue and are restored on return; rejects issuing an already-fully-borrowed book to a second member; verifies zero fine for an on-time return; rejects returning the same transaction twice.

All 20 tests pass. The suite is run with: `javac -d out $(find src/main -name "*.java"), then javac -d testout -cp out $(find src/test -name "*.java"), then java -cp out:testout com.slms.test.TestRunner (or simply ./build.sh test).`

In addition to the automated suite, the application was manually exercised end-to-end via scripted console input covering registration, login, catalog CRUD, issue/return, and every report, confirming correct output at each step (see Section 9).

11. Challenges Faced

- **No internet access to a Maven repository in the build environment** meant JDBC drivers and JUnit could not be pulled in. This was resolved by building persistence directly on java.nio.file CSV I/O behind DAO interfaces, and writing a minimal in-house assertion framework for testing.
- **Reconstructing polymorphic types from flat CSV rows** - since a User row can represent an Admin, Librarian, or Member, UserDAOImpl needed a small factory-style switch on the stored role column to rebuild the correct subclass on every read.
- **Keeping available-copy counts consistent** across concurrent-looking operations (issue, return, and manual copy-count updates) required funnelling every mutation through BookService so invariants ($\text{available} \leq \text{total}$) are enforced in one place.
- **Designing a CSV schema resilient to commas in free-text fields** (book titles, categories) - addressed with a simple escaping strategy that replaces embedded commas before writing.

12. Learnings & Key Takeaways

- Layering an application strictly (UI / Service / DAO) makes it far easier to test business logic in isolation - the entire service layer was unit-tested without ever touching a real file.
- Design patterns earn their complexity when they solve a real, recurring problem in the code - Singleton for shared resources, Factory for polymorphic construction, and DAO for swappable persistence all had a concrete justification in this project rather than being applied for their own sake.
- Checked exceptions, used deliberately for expected failure modes (not found, duplicate, invalid input), produce much friendlier user-facing error handling than letting runtime exceptions propagate.
- Building a small dependency-free test harness deepened understanding of what a framework like JUnit actually automates under the hood.

13. Future Enhancements

- Migrate persistence from CSV files to a real relational database via JDBC, swapping only the DAO implementations.
- Add a password-change / "forgot password" flow for all roles.
- Build a graphical (JavaFX or web) front end on top of the existing service layer, which was designed independently of the console UI.
- Add email/SMS due-date reminder notifications.
- Support multiple library branches and inter-branch transfers.
- Introduce online fine payment via a real payment gateway integration.
- Adopt JUnit/Mockito once the build environment has package-repository access, for a more standard test-reporting workflow.

14. References

- Oracle. *The Java Tutorials*. <https://docs.oracle.com/javase/tutorial/>
- Oracle. *Java Platform, Standard Edition Documentation* (java.nio.file, java.time, java.security packages). <https://docs.oracle.com/en/java/javase/>
- Gamma, Helm, Johnson, Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software* (Singleton, Factory Method, and DAO-adjacent patterns).
- Course lecture material and assignment brief: VITYarthi - *Build Your Own Project, General Project Instructions & Submission Guidelines*.